

Aufgabe 1 (Apriori 1):

In dieser Übung arbeiten Sie mit dem Apriori-Algorithmus, um häufige Itemsets in Transaktionsdaten zu finden. Zuerst sollten sie die Bibliotheken `pandas` und `mlxtend` installieren und importieren gefolgt von ein paar ersten Schritte mit der Bibliothek.

```
In [ ]: %pip install mlxtend pandas
```

```
In [ ]: import pandas as pd
        from mlxtend.preprocessing import TransactionEncoder
```

Jetzt benutzen wir den `TransactionEncoder` um die folgende List in einen korrekten Datensatz zu verwandeln.

```
In [ ]: dataset = [
        ["Apple", "Beer", "Rice", "Chicken"],
        ["Apple", "Beer", "Rice"],
        ["Apple", "Beer"],
        ["Apple", "Bananas"],
        ["Milk", "Beer", "Rice", "Chicken"],
        ["Milk", "Beer", "Rice"],
        ["Milk", "Beer"],
        ["Apple", "Bananas"],
    ]
    te = TransactionEncoder()
    te_ary = te.fit(dataset).transform(dataset)
    te_ary
```

1a) Beschreiben Sie die Ausgabe des vorherigen Blocks, was beinhaltet `te_ary` ?

IHRE ANTWORT:

Wir wandeln `te_ary` nun in ein pandas DataFrame um.

```
In [ ]: df = pd.DataFrame(te_ary, columns=te.columns_)
        df.head()
```

1b) Beschreiben Sie die Ausgabe des vorherigen Blocks, was beinhaltet `df` ?

IHRE ANTWORT:

Mehr Information können Sie unter:

https://rasbt.github.io/mlxtend/user_guide/preprocessing/TransactionEncoder/ finden.

```
In [ ]: from mlxtend.frequent_patterns import apriori

        apriori(df, min_support=0.6, use_colnames=True)
```

1c) Beschreiben Sie die Ausgabe des vorherigen Blocks, was macht die funktion `apriori` ? Welche Bedeutung haben die Parameter?

IHRE ANTWORT:

```
In [ ]: frequent_itemsets = apriori(df, min_support=0.6, use_colnames=True)
frequent_itemsets["length"] = frequent_itemsets["itemsets"].apply(lambda x: len(frequent_itemsets
```

1d) Beschreiben Sie die Ausgabe des vorherigen Blocks, wie unterscheidet sich die Ausgabe von der vorherigen?

IHRE ANTWORT:

Aufgabe 2 (Apriori 2):

Laden Sie den Datensatz `shopping.csv` in Ihr Notebook und machen Sie sich mit dem Datensatz vertraut. Wie oft wird ein Produkt relativ zur Anzahl aller Transaktionen gekauft? Welche Kombinationen von Produkten werden am häufigsten zusammen gekauft (filtern Sie Warenkörbe der Größe eins heraus)? Verwenden Sie hierfür die Bibliothek `mlxtend`. Hinweis: Konvertieren Sie das DataFrame in eine Liste von (Warenkorbs)-Listen.

2a) Laden des Datensatzes

```
In [ ]: df = pd.read_csv("shopping.csv")
# ....
```

2b) Absolute Häufigkeit: Wie oft wurde jedes Produkt gekauft?

```
In [ ]: # absolute_frequency = ...
```

2c) Relative Häufigkeit: Anteil der Transaktionen, in denen ein Produkt gekauft wurde

```
In [ ]: # total_transactions = ...
```

2d) Ermittlung der häufigsten Produktkombinationen mit mindestens 2 Items

```
In [ ]: # frequent_itemsets = ...
```

2e) Sortierung der häufigsten Kombinationen nach Support

```
In [ ]: # frequent_itemsets = ...
```

Aufgabe 3 (K-Means Clustering eines Früchte-Datensatzes)

3a) Laden Sie den `fruits.csv` Datensatz und untersuchen Sie die enthaltenen Merkmale (Länge, Gewicht, Fruchtfleischfarbe, Geschmack) statistisch.

```
In [ ]: df = pd.read_csv("fruits.csv")
# ...
```

3b) Transformieren Sie die Fruchtfleischfarbe und den Geschmack in sinnvolle Werte. Verwenden Sie beim Geschmack dafür die Methode `parse_parts`. Interpretieren Sie die Farbe als Farbwert (auch Farbwinkel), Farbsättigung und Helligkeit.

Beginnen Sie mit der Geschmack-Transformation:

```
In [ ]: import re

weights = {
    "leicht": 0.5,
    "mild": 0.7,
    "sehr": 1.5,
    "stark": 2.0,
    "extrem": 2.5
}

tastes = ["süß", "sauer", "bitter", "herb", "erdig", "scharf", "salzig", "cremig"]

def parse_parts(text):
    result = dict.fromkeys(tastes, 0)

    # splitte nach Komma oder Bindestrich
    parts = re.split(r"[,\-]", text.replace("säuerlich", "sauer"))

    for part in parts:
        part = part.strip()

        weight = 1 # default

        # Modifier prüfen
        for mod, val in weights.items():
            if mod in part:
                weight = val
                part = part.replace(mod, "").strip()

        # Geschmack prüfen
        for taste in tastes:
            if taste in part:
                result[taste] = max(result[taste], weight)

    return pd.Series(result)

# -----
# Ab hier ihr Code:
```

Transformation der Farbwerte:

```
In [ ]: # hue in 360°
# df["h"] =
# df["s"] =
# df["v"] =
```

```
In [ ]: dfinal = df.drop(columns=["Geschmack", "Fruchtfleischfarbe Rot", "Fruchtfleischf
dfinal
```

3c) Wenden Sie den K-Means-Algorithmus aus der scikit-learn Bibliothek an und clustern Sie den Datensatz in fünf Cluster. Visualisieren Sie die gefundenen Cluster in einem 2D-Scatterplot unter Verwendung von zwei Merkmalen. Färben Sie die Punkte entsprechend den gefundenen Clustern ein und vergleichen Sie die Clusterzentren mit den tatsächlichen Gruppen der Früchte. Unterscheiden sich die Clusterzentren signifikant?

```
In [ ]: from sklearn.cluster import KMeans
        from sklearn.preprocessing import StandardScaler

        feature_cols = ["Gewicht (g)", "Länge (cm)", "süß", "sauer", "bitter", "herb", "

        dfinal["h_rad"] = np.deg2rad(dfinal["h"])
        dfinal["h_sin"] = np.sin(dfinal["h_rad"])
        dfinal["h_cos"] = np.cos(dfinal["h_rad"])

        y = dfinal["Gruppe"]
        X = dfinal[feature_cols]

        # -----
        # Ab hier ihr Code für KMeans:
```

```
In [ ]: # -----
        # Ihr Code für die Visualisierung der Clusterzentren und gefärbten Werte zweier
```

3d) Diskutieren Sie, ob der K-Means-Algorithmus für diesen Datensatz geeignet ist, um die vier Früchte-Klassen zu unterscheiden. Erkennen Sie Schwächen oder Besonderheiten in den gefundenen Clustern?

IHRE BESCHREIBUNG HIER

Aufgabe 4 (STL)

4a) Laden Sie den Datensatz `co2.csv` und ersetzen Sie den bestehenden Index durch einen neuen Datumsindex, der im Januar 1959 beginnt und monatlich fortlaufend ist. Visualisieren Sie anschließend den Verlauf der Daten in einem geeigneten Diagramm.

```
In [ ]: co2_df = pd.read_csv('co2.csv')
        # -----
        # Ab hier ihr Code:
```

4b) Erläutern Sie anhand des Graphen, welche Saisonalität Sie für eine STL empfehlen.

IHRE BESCHREIBUNG HIER

4c) Erstellen Sie eine STL-Zerlegung der Zeitreihe mit dem Modul `statsmodels`. Variieren Sie anschließend den Parameter `period`, um zu zeigen, wie sich eine ungeeignete Wahl auf das Ergebnis auswirkt.

In []:

In []:

4d) Begründen Sie, warum Ihre Wahl der Periodenlänge angemessen ist. Erläutern Sie die Bedeutung und Eigenschaften der Komponenten Trend, saisonale Komponente und Residuen im Rahmen der STL-Zerlegung.

IHRE BESCHREIBUNG HIER